

- 428–439.
- [49] Reudismam Rolim, Gustavo Soares, Loris D’Antoni, Oleksandr Polozov, Sumit Gulwani, Rohit Gheyi, Ryo Suzuki, and Björn Hartmann. 2017. Learning syntactic program transformations from examples. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 404–415.
 - [50] Mahesh Kumar Thota, Francis H Shajin, P Rajesh, et al. 2020. Survey on software defect prediction techniques. *International Journal of Applied Science and Engineering* 17, 4 (2020), 331–344.
 - [51] Dániel Varró. 2006. Model Transformation by Example. In *Model Driven Engineering Languages and Systems, 9th International Conference, MoDELS 2006, Genova, Italy, October 1-6, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4199)*, Oscar Nierstrasz, Jon Whittle, David Harel, and Gianna Reggio (Eds.). Springer, 410–424. https://doi.org/10.1007/11880240_29
 - [52] Dániel Varró and Zoltán Balogh. 2007. Automating model transformation by example using inductive logic programming. In *Proceedings of the 2007 ACM Symposium on Applied Computing (SAC), Seoul, Korea, March 11-15, 2007*, Yookun Cho, Roger L. Wainwright, Hisham Haddad, Sung Y. Shin, and Yong Wan Koo (Eds.). ACM, 978–984. <https://doi.org/10.1145/1244002.1244217>
 - [53] Yanlin Wang, Kefeng Duan, Dewu Zheng, Ensheng Shi, Fengji Zhang, Yanli Wang, Jiachi Chen, Xilin Liu, Yuchi Ma, Hongyu Zhang, Qianxiang Wang, and Zibin Zheng. 2025. Towards an Understanding of Context Utilization in Code Intelligence. arXiv:2504.08734 [cs.SE] <https://arxiv.org/abs/2504.08734>
 - [54] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE symposium on security and privacy*. IEEE, 590–604.
 - [55] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. 2024. On the Evaluation of Large Language Models in Unit Test Generation. arXiv:2406.18181 [cs.SE] <https://arxiv.org/abs/2406.18181>
 - [56] Michael Zhivich and Robert K Cunningham. 2009. The real cost of software errors. *IEEE Security & Privacy* 7, 2 (2009), 87–90.
 - [57] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* (2024).

A DEFS Construction Details

A.1 Construction Process

We constructed DEFS through three main stages.

Commit Collection. We first mined defect-fixing commits from GitHub. To ensure both diversity and practical relevance, we filtered projects based on two criteria: (1) more than 5,000 stars, and (2) containing Java code. This configuration yielded a total of 488 projects, each exhibiting active development and a rich history of real defect fixes. We restricted commits to the period between January 1, 2023 and January 1, 2025, and removed non-code changes such as documentation updates. Using the keywords “fix” and “repair” in commit messages, we collected 95,335 candidate commits from 102 popular repositories.

Checker-Based Identification. The initial keyword search using commit messages (“fix,” “repair”) served only as a coarse-grained entry point and was not intended to directly yield usable defect-fix data. To obtain reliable instances, we applied two complementary static analyzers: CodeQL (138 defect-related queries) and PMD (147 rules). For each commit in the candidate set, we executed the corresponding analyzers on both the parent and the fixed revision. If a checker-reported warning disappeared in the fixed revision, the commit was considered a potential defect fix for that defect. From such commits, we extracted method-level code changes by differencing the defective and fixed versions, associating each pair with the checker that identified it. This process produced method-level defective–fix pairs, which were grouped according to the checker that identified them and later used for clustering. Consistent with the intra-procedural focus of our approach, we consider only code changes confined within a single method during dataset construction. Under this setting, PMD produces 31,598 changed warnings, among which 10,606 (about 34%) correspond to method-level code changes, and CodeQL identifies 9,663 changed warnings, of which 7,890 (about 82%) are confined to a single method.

Grouping and Validation. To organize similar defect–fix pairs, we grouped defect–fix pairs into clusters using a hierarchical clustering algorithm that combines checker labels with edit-structure similarity. To ensure validity, the first two authors independently annotated all clusters after aligning on annotation criteria, achieving substantial agreement (Cohen’s kappa ≈ 0.8). Disagreements were resolved through discussion.

After this process, the final dataset consists of 1,287 defect–fix pairs organized into 319 clusters, covering 63 defect categories. Further statistics on distribution and diversity are presented in the next section.

Defect-Inducing Commit Identification (RQ2). For RQ2, we further identify the defect-inducing commit for each defect–fix pair following the SZZ algorithm. Specifically, we apply `git blame` to the modified lines in the defective version to trace back to the commits that last modified those lines. We then evaluate these commits in reverse chronological order using the corresponding static checker until the defect-inducing commit is identified.

A.2 Dataset Statistics

Table 5. Top-5 checkers per subset in DEFS. In this table, #Clus. denotes the number of clusters for each checker, #Pairs denotes the number of defect–fix pairs for each checker, and the Avg. Size denotes the average number of defect–fix pairs per cluster.

CodeQL				PMD			
Checker	#Clus.	#Pairs	Avg. Size	Checker	#Clus.	#Pairs	Avg. Size
Ineff. primitive constructor	14	52	3.71	Use collection is empty	18	88	4.89
Deprecated method/ctor invocation	10	31	3.10	JUnit5 test pkg private	18	64	3.56
Missing Override annotation	5	28	5.60	Primitive wrapper init	17	74	4.35
Missing catch of NumberFormatException	4	9	2.25	JUnit use expected	17	53	3.12
Expression always true	3	30	10.00	System println	14	75	5.36

To highlight the diversity and representativeness of DEFS, we provide additional statistics on its composition. Table 5 reports the top-5 checkers from each subset (CodeQL and PMD), showing both the number of clusters and the total defect–fix pairs they contribute. The CodeQL subset emphasizes defect-oriented categories such as *missing exception handling* and *always-true expressions*, while the PMD subset covers widely recurring unit-test and best-practice rules (e.g., *JUnit conventions*, *logging*, *system output*). This complementary coverage ensures that the dataset is not biased toward a single defect type and reflects both semantic defects and developer practice issues that frequently arise in real-world projects.

Fig. 6a shows the cluster size distribution in DEFS. Most clusters are small: 59% contain only 2–3 examples, and another 21.3% contain 4–5 examples. Mid-sized clusters (6–10 examples) account for 15%, while large clusters (10+ examples) make up 3.8%. This long-tail distribution indicates that the dataset captures both rare, fine-grained fixes and highly recurring defect patterns, providing realistic coverage of how defects appear in practice. The prevalence of small clusters further underscores the need for methods that can abstract meaningful patterns from few-shot examples—an ability that DSLGEN is explicitly designed to provide.

Finally, we examine the edit scope of defect–fix pairs. As the results shown in Fig. 6b, most fixes are concise: about 66% of pairs involve no more than five changed lines. Nevertheless, 18% include 6–10 lines, 10% require 11–20 lines, and nearly 6% exceed 20 lines. This distribution confirms that DEFS reflects not only the common short edits typical of defect fixes, but also medium and large modifications, thereby offering diversity in modification complexity that is crucial for evaluating generalizable defect patterns.

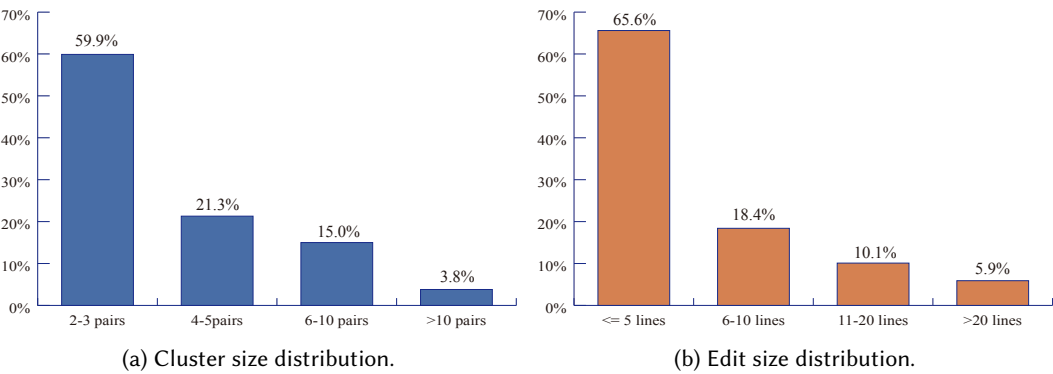


Fig. 6. Dataset statistics for DEFS